

You are a senior TypeScript engineer and production software architect.

Build a completely independent Facebook Page automation service for **MobDeals Kenya** using only:

- Cloudflare Workers Free Tier
- Cloudflare Cron Triggers
- Cloudflare Queues where available
- Supabase Free Tier
- Supabase PostgreSQL
- Supabase Storage
- Meta Graph API
- Meta Webhooks
- Meta Business Suite

The service must not depend on an existing website, Astro project, Render, n8n or any external paid hosting service.

1. Main objective

Build a standalone service that can:

1. Automatically publish text and image posts to a Facebook Page.
2. Schedule posts using Supabase database records.
3. Receive Facebook Page comment webhooks.
4. Reply to selected comments using approved rules.
5. Receive Facebook Messenger webhook events.
6. Send safe rule-based Messenger replies.
7. Save comments, conversations and leads in Supabase.
8. Escalate important conversations to Meta Business Suite for human handling.
9. Record logs, failures and audit history.
10. Operate within Cloudflare and Supabase free-tier limitations.

There will be no public website and no custom admin dashboard in the first version.

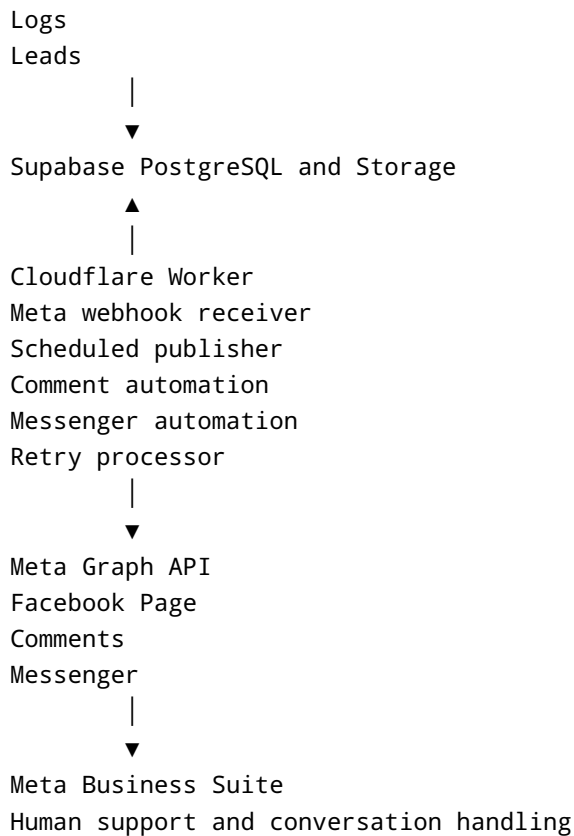
Use:

- **Supabase Studio** to manage products, posts, reply templates, leads and automation settings.
- **Meta Business Suite** for human replies to comments and Messenger conversations.
- **Cloudflare Workers** as the automation engine.

2. Required architecture

Use this architecture:

```
Supabase Studio
Products
Post schedules
Reply templates
Automation settings
```



The service must operate independently.

Do not create:

- A public website.
- An Astro application.
- A customer-facing dashboard.
- A custom Messenger inbox.
- A Render deployment.
- An n8n dependency.
- A payment system.

3. Administration approach

Since there is no website or custom dashboard, use Supabase Studio as the first-release administration interface.

Administrators must be able to manage the following through Supabase Studio:

- Products.
- Prices.
- Stock status.
- Product images.
- Facebook post drafts.
- Post approval status.
- Scheduled posting dates.

- Reply templates.
- Comment automation rules.
- Messenger automation rules.
- Automation switches.
- Failed jobs.
- Leads.
- Conversation summaries.
- Human-handoff status.

Human replies must be completed through Meta Business Suite.

Do not attempt to recreate Meta Business Suite.

4. Technology requirements

Use:

- TypeScript.
- Cloudflare Workers.
- Wrangler.
- Cloudflare Cron Triggers.
- Cloudflare Queues when suitable.
- Supabase PostgreSQL.
- Supabase Storage.
- Meta Graph API.
- Meta Webhooks.
- Zod for runtime validation.
- Vitest for automated testing.

Use strict TypeScript.

Avoid unnecessary packages.

The application must be designed for the Cloudflare Workers runtime and must not depend on Node.js-only APIs.

5. Suggested project structure

Use a structure similar to:

```
facebook-automation/  
  src/  
    index.ts  
    config/  
    routes/  
    meta/  
    webhooks/  
    publishing/  
    comments/  
    messenger/
```

```
products/  
leads/  
database/  
queue/  
retries/  
security/  
logging/  
validation/  
utilities/  
types/  
migrations/  
tests/  
docs/  
wrangler.toml  
package.json  
tsconfig.json  
vitest.config.ts  
.env.example  
README.md
```

Adjust the structure where necessary while preserving clear separation of responsibilities.

6. Environment variables

Support the following configuration:

```
SUPABASE_URL  
SUPABASE_SERVICE_ROLE_KEY  
  
META_APP_ID  
META_APP_SECRET  
META_PAGE_ID  
META_PAGE_ACCESS_TOKEN  
META_VERIFY_TOKEN  
META_GRAPH_API_VERSION  
  
INTERNAL_ADMIN_SECRET  
ENVIRONMENT  
LOG_LEVEL
```

Store production secrets using Cloudflare Worker secrets.

Never commit real credentials.

Never expose the Supabase service-role key or Meta access token in logs or API responses.

Create a complete `.env.example` containing placeholders only.

7. Required Worker routes

Implement the following public routes:

```
GET /webhooks/meta
POST /webhooks/meta
GET /health
```

GET /webhooks/meta

Use this endpoint for Meta webhook verification.

It must:

- Read the verification mode.
- Read the verification token.
- Compare it with the configured token.
- Return the challenge when valid.
- Reject invalid requests.
- Avoid leaking configuration values.

POST /webhooks/meta

Use this endpoint for Meta webhook deliveries.

It must:

- Read the raw request body.
- Verify the Meta request signature.
- Reject invalid signatures.
- Parse the webhook payload safely.
- Validate the payload structure.
- Store the raw event in Supabase.
- Prevent duplicate processing.
- Send the event to a queue or processing workflow.
- Return a successful response quickly.
- Avoid performing long operations before acknowledging Meta.

GET /health

Return a minimal health response.

Do not expose secrets, database credentials, access tokens or internal configuration.

8. Optional protected administrative endpoints

Create protected internal endpoints for testing and operations:

```
POST /internal/posts/:id/publish
POST /internal/events/:id/retry
POST /internal/automation/pause
POST /internal/automation/resume
GET /internal/status
```

Protect them using a strong internal secret or signed request.

These endpoints are for controlled testing and emergency operations.

Do not expose them without authentication.

9. Supabase database design

Create SQL migrations for tables similar to:

```
products
product_images
facebook_pages
facebook_posts
meta_webhook_events
facebook_comments
comment_replies
messenger_contacts
messenger_conversations
messenger_messages
reply_templates
reply_rules
leads
automation_settings
automation_failures
audit_logs
```

Improve or normalize this design where necessary.

Products

Each product should support:

- UUID.
- Product name.
- Brand.
- Model.
- Category.
- Processor.
- RAM.
- Storage.
- Graphics.

- Display.
- Description.
- Price.
- Currency.
- Stock quantity.
- Stock status.
- Product condition.
- Active status.
- Created timestamp.
- Updated timestamp.

Supported stock states should include:

```
in_stock
low_stock
out_of_stock
reserved
unknown
```

Product images

Store:

- Product reference.
- Supabase Storage path.
- Public URL where appropriate.
- MIME type.
- File size.
- Primary image indicator.
- Active status.

Facebook posts

Each scheduled post should support:

- UUID.
- Product reference.
- Caption.
- Image URL.
- Post type.
- Status.
- Scheduled time.
- Published time.
- Meta Page ID.
- Meta post ID.
- Approval status.
- Retry count.
- Last error.
- Processing lock.
- Created timestamp.

- Updated timestamp.

Supported post types:

```
text
single_image
```

Supported post states:

```
draft
pending_approval
approved
scheduled
processing
published
failed
cancelled
```

Webhook events

Store:

- External event identifier.
- Event category.
- Raw payload.
- Processing status.
- Received time.
- Processed time.
- Retry count.
- Last error.
- Next retry time.

Supported event states:

```
received
queued
processing
completed
failed
ignored
```

Comments

Store:

- Meta comment ID.
- Meta post ID.
- Commenter ID.

- Commenter name where permitted.
- Comment text.
- Detected intent.
- Reply status.
- Human-handoff status.
- Created time.

Messenger conversations

Store:

- Meta sender identifier.
- Conversation state.
- Last customer interaction time.
- Automation enabled status.
- Human-handoff status.
- Collected sales information.
- Last intent.
- Created timestamp.
- Updated timestamp.

Leads

Store:

- Customer identifier.
- Customer name where available.
- Product interest.
- Budget.
- RAM preference.
- Storage preference.
- Location.
- Phone number when voluntarily supplied.
- Source.
- Conversation reference.
- Follow-up status.
- Assigned staff member.
- Created timestamp.

10. Database protections

Implement:

- UUID primary keys.
- Foreign keys.
- Status constraints.
- Uniqueness constraints.
- Suitable indexes.
- Updated timestamp triggers.
- Duplicate-event protections.

Add uniqueness protections for:

- Meta webhook event IDs.
- Meta post IDs.
- Meta comment IDs.
- Meta Messenger message IDs.
- Comment replies.
- Scheduled publishing attempts.

Create indexes for:

- Posts scheduled for publication.
- Unprocessed webhook events.
- Failed jobs requiring retry.
- Open conversations.
- Leads requiring follow-up.
- Meta post lookups.
- Meta user lookups.

11. Supabase Row Level Security

Enable Row Level Security where appropriate.

Requirements:

- Public users must not access any tables.
- The Cloudflare Worker must use the Supabase service-role key.
- The service-role key must only exist inside Cloudflare secrets.
- No browser client will access Supabase directly.
- Conversations, messages, leads and logs must remain private.
- Supabase Studio access will be controlled through the Supabase project account.

Document why the Worker uses the service-role credential.

12. Scheduled Facebook publishing

Use a Cloudflare Cron Trigger to check for due posts.

The workflow should be:

```
Create product in Supabase
→ Upload image to Supabase Storage
→ Create Facebook post record
→ Set status to approved
→ Set scheduled date and time
→ Cron finds due post
→ Lock post
→ Publish to Facebook
```

- Store Meta post ID
- Mark as published

Use `Africa/Nairobi` for scheduling input.

Store all database timestamps in UTC.

Before publishing, verify:

- Automation is enabled.
- Posting automation is enabled.
- Post status is approved or scheduled.
- The scheduled time has arrived.
- The post has not already been published.
- The product is active.
- The product is in stock.
- The caption is not empty.
- The image exists for image posts.
- The Page access token is configured.

Prevent two Cron executions from publishing the same post.

Use safe database locking or an atomic database update.

After successful publishing:

- Save the Meta post ID.
- Save the publication time.
- Save relevant non-sensitive response metadata.
- Mark the record as published.
- Create an audit log.

After failure:

- Classify the error.
- Save the failure.
- Increment the retry count.
- Schedule a retry where appropriate.
- Mark non-retryable failures for human review.

13. Posting rules

The first release must support:

- Text-only Page posts.
- Single-image Page posts.
- Product-linked posts.
- Scheduled publication.
- Manual publication using a protected endpoint.
- Cancellation before publication.

Do not support initially:

- Video uploads.
- Multiple-image carousel posts.
- Facebook Stories.
- Facebook Reels.
- Paid ads.
- Boosted posts.
- Cross-posting.
- Automatic reposting.

Design the service so these may be added later.

14. Image handling

Use Supabase Storage for images.

The system must:

- Validate image URLs.
- Validate MIME type.
- Reject unsupported formats.
- Enforce a configurable maximum file size.
- Confirm the image is accessible before publication.
- Record failures caused by inaccessible images.

Support:

```
image/jpeg  
image/png  
image/webp
```

Do not generate or edit images.

Images will be uploaded manually through Supabase Storage.

15. Comment webhook automation

Receive Facebook Page comment events through Meta webhooks.

Store every valid comment before deciding whether to reply.

Detect common intents:

```
price  
availability  
location  
delivery
```

```
specifications
inbox_request
human_support
complaint
unknown
```

Support wording such as:

```
Price?
How much?
Available?
Still available?
Where are you located?
Do you deliver?
Specs?
Inbox me.
I need assistance.
```

The comment automation must:

- Match the Facebook post to the Supabase post record.
- Match the post to a product.
- Retrieve the latest product data.
- Use approved reply templates.
- Avoid inventing product information.
- Reply only once per person, rule and post.
- Ignore comments created by the Page itself.
- Ignore duplicate webhook events.
- Prevent recursive reply loops.
- Escalate unclear or sensitive comments.
- Avoid asking for phone numbers publicly.

16. Comment replies

Safe automatic replies may include:

Price

```
The current price for {{product_name}} is KSh {{price}}. Please message us
privately to confirm the latest availability.
```

Availability

```
{{product_name}} is currently marked as {{availability}}. Please send us a
private message so we can confirm before you visit or make payment.
```

Delivery

We offer delivery within Nairobi and countrywide. Please send your location privately for the available delivery options.

Location

Please send us a private message and our team will share the current shop location and directions.

Human assistance

Thank you for contacting MobDeals Kenya. A member of our team will assist you directly.

Store these as database templates rather than hard-coded permanent responses.

17. Comments requiring human review

Never automatically resolve:

- Payment disputes.
- Refund requests.
- Warranty claims.
- Fraud accusations.
- Legal complaints.
- Threats.
- Abusive messages.
- Complaints about staff.
- Delivery disputes.
- Missing product data.
- Requests involving sensitive personal information.

Mark these as requiring human review.

Human staff will respond through Meta Business Suite.

18. Messenger webhook automation

Receive and store Messenger webhook events.

Implement a simple rule-based sales flow.

Suggested flow:

Customer sends message

- Greeting
- Ask which product they need
- Ask their budget
- Ask preferred specifications
- Search active Supabase products
- Present matching products
- Offer human assistance

Collect only useful sales information:

- Product category.
- Model.
- Budget.
- RAM preference.
- Storage preference.
- Location.
- Delivery preference.
- Phone number only when voluntarily supplied.

The bot must:

- Respect Meta messaging limitations.
- Store the customer's latest interaction time.
- Avoid automated promotional messages outside allowed conditions.
- Stop automation when human handoff is activated.
- Avoid negotiating prices.
- Avoid confirming payments.
- Avoid confirming orders.
- Avoid promising delivery dates.
- Avoid claiming unavailable products are available.
- Escalate unclear questions.

19. Product matching

Implement basic rule-based product matching.

The matching process may use:

- Product category.
- Brand.
- Price range.
- RAM.
- Storage.
- Processor.
- Stock status.

Only return products that are:

- Active.

- In stock or low stock.
- Within the customer's budget where supplied.

Limit automatic suggestions to a small configurable number, such as three products.

Do not use AI for the first release.

20. Human handoff

Conversation states should include:

```
automation_active
human_required
human_assigned
resolved
closed
```

Trigger human handoff when:

- The customer asks for a person.
- A complaint is detected.
- A payment issue is mentioned.
- A warranty issue is mentioned.
- The customer appears dissatisfied.
- The bot cannot determine the request.
- Required product information is missing.
- The same question is repeated.
- The reply limit has been reached.

When human handoff is active:

- Do not send further automatic replies.
- Save the reason.
- Save the handoff time.
- Leave the conversation available for Meta Business Suite handling.

21. Reply templates

Create a `reply_templates` table.

Each template must support:

- Name.
- Intent.
- Channel.
- Body.
- Language.
- Active status.
- Approval status.

- Version.
- Variables.
- Created timestamp.
- Updated timestamp.

Supported channels:

```
facebook_comment  
messenger
```

Supported variables may include:

```
customer_name  
product_name  
price  
availability  
location  
delivery_summary
```

Validate all variables before sending.

If a required variable is missing, do not send an incomplete template.

Escalate the conversation instead.

22. Automation controls

Create database settings for:

```
global_automation_enabled  
posting_enabled  
comment_replies_enabled  
messenger_replies_enabled  
maintenance_mode  
maximum_daily_posts  
maximum_comment_replies_per_post  
maximum_replies_per_customer  
maximum_messenger_replies_per_conversation
```

The Worker must check these settings before taking automated action.

Webhook ingestion must continue even when replies are disabled.

A global emergency switch must stop all outbound automation while still recording incoming events.

23. Idempotency

Meta may deliver the same event more than once.

Implement strong idempotency.

The service must prevent:

- Duplicate post publication.
- Duplicate comment replies.
- Duplicate Messenger replies.
- Duplicate lead records.
- Duplicate webhook processing.

Use external Meta identifiers and database uniqueness constraints.

Document the idempotency strategy.

24. Queue processing

The webhook endpoint must respond quickly.

Preferred workflow:

```
Receive webhook
→ Verify signature
→ Validate payload
→ Store event
→ Add event to Cloudflare Queue
→ Return success
→ Queue consumer processes event
```

If Cloudflare Queues are unavailable or unsuitable in the selected free-tier environment, implement a safe database-backed event queue using Supabase.

Do not discard failed events.

Store every failed event for review.

25. Retry handling

Implement exponential backoff for retryable failures.

Retryable failures may include:

- Temporary Meta API failure.
- Network timeout.
- Rate limiting.

- Temporary Supabase error.
- Cloudflare transient error.

Non-retryable failures may include:

- Invalid Meta token.
- Missing Meta permission.
- Invalid Page ID.
- Invalid image URL.
- Deleted Facebook content.
- Invalid database record.
- Validation failure.

Store:

- Retry count.
- Last retry time.
- Next retry time.
- Error code.
- Error category.
- Redacted error message.
- Human-review status.

Set a maximum retry count.

26. Logging

Use structured JSON logs.

Include where relevant:

- Request ID.
- Webhook event ID.
- Meta object ID.
- Facebook post ID.
- Comment ID.
- Conversation ID.
- Operation.
- Result.
- Duration.
- Error category.

Never log:

- Meta access tokens.
- Supabase service-role keys.
- App secrets.
- Authorization headers.
- Full phone numbers.
- Unnecessary private message content.

Mask sensitive customer details.

27. Audit logs

Record database audit events for:

- Post creation.
- Post approval.
- Post scheduling.
- Post publication.
- Post cancellation.
- Automation pause.
- Automation resume.
- Reply-template changes.
- Manual retry.
- Human handoff.
- Conversation resolution.
- Setting changes.

28. Security

Implement:

- Meta request-signature verification.
- Runtime validation of all external input.
- Authentication for internal endpoints.
- Secret storage through Cloudflare.
- Error redaction.
- Rate limiting for internal routes.
- Safe database access.
- Minimal public endpoints.
- No browser-accessible service-role credentials.

Treat all webhook data as untrusted.

Do not trust Meta payload fields without validation.

29. Privacy

Handle customer information responsibly.

Requirements:

- Store only information required for customer support and sales.
- Do not store unnecessary sensitive data.
- Mask phone numbers in logs.
- Provide a configurable retention period.
- Allow records to be anonymized.
- Allow leads and conversations to be deleted.
- Keep comments separate from private Messenger data.
- Never expose private customer information publicly.
- Do not use customer conversations for unrelated marketing.

30. Free-tier safeguards

Design the system to conserve free-tier resources.

Implement:

- Pagination.
- Limited database queries.
- Indexed scheduled-job searches.
- Small webhook payload records where possible.
- Configurable log retention.
- Configurable message retention.
- Bounded retries.
- Daily posting limits.
- Reply-frequency limits.
- No heavy AI processing.
- No image generation.
- No unnecessary polling.
- Cron execution only at a reasonable interval.
- Batch processing of due posts and retry jobs.

Avoid designs requiring an always-running server.

Use event-driven Cloudflare Workers.

31. Testing

Create tests for:

- Successful Meta webhook verification.
- Invalid webhook verification token.
- Valid Meta request signature.
- Invalid Meta request signature.
- Malformed webhook payload.
- Duplicate webhook event.
- Duplicate comment.
- Duplicate Messenger message.
- Duplicate scheduled-post execution.
- Text-post publication.
- Image-post publication.
- Missing image.
- Inaccessible image.
- Invalid Meta token.
- Missing Meta permission.
- Meta rate limiting.
- Product matching.
- Out-of-stock product handling.
- Comment intent detection.
- Complaint escalation.
- Human-handoff activation.

- Automation pause.
- Template rendering.
- Missing template variables.
- Retryable errors.
- Non-retryable errors.
- Internal endpoint authentication.

Mock all Meta API calls in tests.

Tests must not require real Meta credentials.

32. Documentation

Create:

```
README.md
docs/ARCHITECTURE.md
docs/META_SETUP.md
docs/SUPABASE_SETUP.md
docs/CLOUDFLARE_SETUP.md
docs/SECURITY.md
docs/OPERATIONS.md
docs/DEPLOYMENT.md
docs/INCIDENT_RESPONSE.md
```

Meta setup documentation

Explain:

- How to configure the Meta developer app.
- How to connect the Facebook Page.
- How to set the webhook callback URL.
- How to create the verification token.
- How to configure the Page access token.
- Which webhook subscriptions are required.
- Which permissions may be required.
- How to test while the Meta app is in development mode.
- Which steps may require Meta App Review.
- How to disable automation safely.

Do not claim Meta App Review approval is guaranteed.

Supabase setup documentation

Explain:

- Creating the Supabase project.
- Running migrations.
- Creating the image Storage bucket.
- Uploading product images.

- Creating products.
- Creating scheduled posts.
- Approving posts.
- Managing templates.
- Pausing automation.
- Reviewing failures.
- Reviewing leads.

Cloudflare setup documentation

Explain:

- Creating the Worker.
- Configuring Wrangler.
- Setting secrets.
- Configuring the Cron Trigger.
- Configuring Cloudflare Queues if used.
- Deploying staging.
- Deploying production.
- Viewing logs.
- Rolling back deployment.

33. Deployment environments

Support:

```
development
staging
production
```

Staging must support a safe mode.

In staging mode:

- Store webhook events.
- Do not reply to real customers.
- Do not publish real posts unless explicitly enabled.
- Clearly mark simulated actions.

Never put production secrets in `wrangler.toml`.

34. Initial setup data

Create seed data for:

- Automation settings.
- Basic reply templates.
- Comment intents.
- Messenger conversation states.
- Post statuses.

- One example product.
- One example unpublished Facebook post.

The example data must contain no real credentials or customer information.

35. First-release scope

Include

- Independent Cloudflare Worker project.
- Supabase database.
- Supabase Storage images.
- Meta webhook verification.
- Meta webhook signature validation.
- Scheduled text posting.
- Scheduled single-image posting.
- Comment event storage.
- Conservative comment replies.
- Messenger event storage.
- Basic Messenger sales flow.
- Product matching.
- Human handoff.
- Lead capture.
- Retry handling.
- Audit logging.
- Tests.
- Deployment documentation.

Exclude

- Public website.
- Custom admin dashboard.
- Custom customer inbox.
- n8n.
- Render.
- Artificial intelligence replies.
- AI image generation.
- WhatsApp.
- Instagram.
- Payment processing.
- Order processing.
- Refund processing.
- Automatic price negotiation.
- Facebook advertising management.
- Multi-page or multi-business support.

36. Implementation phases

Implement in this order.

Phase 1: Foundation

- Create the project.
- Configure TypeScript.
- Configure Wrangler.
- Add environment validation.
- Add Supabase client.
- Add logging.
- Create database migrations.

Phase 2: Webhook foundation

- Add webhook verification.
- Add signature verification.
- Add event validation.
- Store events.
- Add duplicate protection.
- Add tests.

Phase 3: Scheduled posting

- Add post workflow.
- Add Cron Trigger.
- Add safe locking.
- Add text posting.
- Add image posting.
- Store Meta post IDs.
- Add retry handling.
- Add tests.

Phase 4: Comment automation

- Store comments.
- Add intent matching.
- Add reply templates.
- Add reply limits.
- Add complaint escalation.
- Add tests.

Phase 5: Messenger automation

- Store contacts.
- Store conversations.
- Add guided sales flow.
- Add product matching.
- Add lead creation.
- Add human handoff.
- Add tests.

Phase 6: Production hardening

- Add structured logs.
- Add audit logs.

- Add privacy controls.
- Add retention guidance.
- Add emergency automation shutdown.
- Complete documentation.
- Complete deployment configuration.

37. Definition of done

The service is complete when:

1. It can be deployed independently to Cloudflare Workers.
2. It does not require a website.
3. Supabase Studio can be used to create and schedule posts.
4. Meta can verify the webhook endpoint.
5. Invalid Meta signatures are rejected.
6. Valid webhook events are stored.
7. Duplicate webhook events are ignored safely.
8. Approved posts publish at the correct scheduled time.
9. Duplicate posts cannot be published.
10. Meta post IDs are stored.
11. Comments are stored.
12. Approved comment rules can send replies.
13. Complaints are escalated.
14. Messenger messages are stored.
15. Basic sales questions can be handled.
16. Matching products can be suggested.
17. Human handoff stops automation.
18. Leads are saved in Supabase.
19. Automation can be paused through Supabase settings.
20. Failed operations are recorded and retryable.
21. Secrets are stored securely.
22. Tests pass without real Meta credentials.
23. Deployment instructions are complete.
24. Human conversations remain manageable through Meta Business Suite.
25. The system operates without Render, n8n or an always-running server.

38. Working instructions

Before writing code:

1. Present the proposed architecture.
2. Present the database design.
3. Present the project folder structure.
4. Identify assumptions.
5. Identify Meta dashboard steps that require manual action.
6. Present the implementation phases.

Then implement the project phase by phase.

During implementation:

- Do not fabricate successful Meta API calls.
- Do not fabricate deployment results.
- Do not expose secrets.
- Do not silently ignore failing tests.
- Use strict runtime validation.
- Keep modules small and maintainable.
- Document important architectural decisions.
- Make reasonable decisions without repeatedly requesting confirmation.
- Clearly identify any step that must be completed manually in Meta, Supabase or Cloudflare.

At completion provide:

1. Architecture summary.
2. Files created.
3. Database tables and migrations.
4. Environment variables.
5. Supabase setup steps.
6. Cloudflare setup steps.
7. Meta configuration steps.
8. Testing instructions.
9. Deployment instructions.
10. Operational instructions.
11. Known limitations.
12. Recommended future improvements.

Start by presenting the architecture and implementation plan.